Unix/Linux System Calls

Explanation of File and Directory System Calls

--Saurav Kumar

Introduction

- Unix/Linux system calls act as an interface between user applications and the kernel.
- They provide functionalities for **file handling, directory management, and file control.**

File I/O System Calls

- The following system calls are used for handling files:

- `open()` – Open a file
- `read()` – Read from a file
- `write()` – Write to a file
- `close()` – Close a file

open( ) – Open a File

- **Syntax:**
- `int fd = open(const char *pathname, int flags, mode_t mode);`
-
- **Example:**
- `int fd = open("example.txt", O_CREAT | O_WRONLY, 0644);`
-
- **Explanation:**
- Opens or creates `example.txt` with write permissions (`0644`).

`read()` – Read from a File

- **Syntax:**
- `ssize_t read(int fd, void *buf, size_t count);`
-
- **Example:**
- `char buffer[100];`
- `ssize_t bytesRead = read(fd, buffer, sizeof(buffer));`
-
- **Explanation:**
- Reads up to 100 bytes into `buffer`

`write()` – Write to a File

- **Syntax:**
- `ssize_t write(int fd, const void *buf, size_t count);`
-
- **Example:**
- `const char *data = "Hello, World!";`
- `ssize_t bytesWritten = write(fd, data, strlen(data));`
-
- **Explanation:**
- Writes "Hello, World!" to the file.

`close()` – Close a File

- **Syntax:**
- `int close(int fd);`
-
- **Example:**
- `close(fd);`
-
- **Explanation:**
- Closes the file.

`fcntl()` – File Control

- **Syntax:**
- `int fcntl(int fd, int cmd, ...);`
-
- **Example:**
- `int flags = fcntl(fd, F_GETFL);`
- `fcntl(fd, F_SETFL, flags | O_APPEND);`
-
- **Explanation:**
- The fcntl (file control) system call is used to perform various operations on file descriptors, such as setting file locks, changing file descriptor flags, and duplicating file descriptors.

`fcntl()` – File Control

- F_GETFD Get file descriptor flags
- F_SETFD Set file descriptor flags
- F_GETFL Get file status flags (e.g., read/write mode)
- F_SETFL Set file status flags (e.g., non-blocking mode)
- F_DUPFD Duplicate a file descriptor
- F_GETLK Get file lock information
- F_SETLK Set (non-blocking) file lock
- F_SETLKW Set (blocking) file lock

`lseek()` – Seek to a Position

- **Syntax:**
- `off_t lseek(int fd, off_t offset, int whence);`
-
- **Example:**
- `lseek(fd, 0, SEEK_END);`
-
- **Explanation:**
- The lseek system call is used to move the file offset (cursor position) within a file descriptor. It is useful for random access to file contents.

`lseek()` – Seek to a Position

- SEEK_SET Set file offset to offset bytes from the beginning.
- SEEK_CUR Move file offset by offset bytes from the current position.
- SEEK_END Move file offset by offset bytes from the end of the file.

`stat()` – Get File Information

- **Syntax:**
- `int stat(const char *pathname, struct stat *buf);`
-
- **Example:**
- `struct stat fileStat;`
- `stat("example.txt", &fileStat);`
- `printf("File size: %ld bytes\n", fileStat.st_size);`
-
- **Explanation:**
- The stat system call is used to retrieve information about a file such as its size, permissions, timestamps, and more.

`opendir()` – Open a Directory

- **Syntax:**
- `DIR *opendir(const char *name);`
-
- **Example:**
- `DIR *dir = opendir("/home/user");`
-
- **Explanation:**
- The opendir function is used to open a directory for reading its contents. It returns a pointer to a DIR structure.

`readdir()` – Read a Directory

- **Syntax:**
- `struct dirent *readdir(DIR *dir);`
-
- **Example:**
- `struct dirent *entry;`
- `while ((entry = readdir(dir)) != NULL) {`
- `    printf("File: %s\n", entry->d_name);`
- `}`
-
- **Explanation:**
- The readdir function is used to read entries from a directory that has been opened using opendir.

Thank You